Winter 2026

ECE 315 Computer Interfacing

LAB 4: UART based Controlled Stepper Motor Using Zybo Z7-10 and FreeRTOS

DATES

Section	Date	Demo Due Date	Report Due Date
H21	24-Mar-2026 2:00 to 4:50 PM	04-Apr-2025 4:50 PM	07-Apr-2026 11:00 PM
H31	25-Mar-2026 2:00 to 4:50 PM	04-Apr-2025 4:50 PM	08-Apr-2026 11:00 PM
H41	26-Mar-2026 2:00 to 4:50 PM	07-Apr-2025 4:50 PM	09-Apr-2026 11:00 PM
H51	27-Mar-2026 2:00 to 4:50 PM	07-Apr-2025 4:50 PM	10-Apr-2026 11:00 PM

LEARNING OBJECTIVES

- To gain more experience running the FreeRTOS real-time kernel.
- To control the operation of a small stepper motor using different drive modes.
- To enhance the user interface of a stepper motor control application on the Zybo Z7 using FreeRTOS.
- To gain experience with measuring experimentally the speed and acceleration limits of a small stepper motor.

HARDWARE PLATFORM AND SOFTWARE ENVIRONMENT

- Once again, the controller platform is the Digilent Zybo Z7 development board. As with all the previous labs, CPU0 in the Zynq-7000 SoC will run the FreeRTOS real-time kernel.
- The fixed Zynq-7000 SoC hardware platform and the initial stepper motor control software for Part 1 must be downloaded from the Lab 4 section of the lab Canvas site. The given software contains the initial application code in C that you will be modifying.
- The Zybo Z7 board will be augmented with one 28BYJ-48 stepper motor with unipolar drive windings, one 6-VDD power supply, one ULN2003-based driver module, one breadboard, one LTV-847 opto-isolator transistor array, four 220 Ω resistors, four 10 kΩ resistors, and wires.
- You will be provided with two setup files: one for the hardware and one for the software setup (check the links on the resources section), that will provide you with detailed instructions on how to set up your project.

Note: The stepper motor must be powered by the DC power supply and NOT by the Zybo Z7 board. The power supply's voltage and the voltage spikes on the motor windings can readily damage the digital circuits on the Zybo Z7 board.

PRE LAB 10%

1. Read the source files to get an understanding of the application.
2. Draw an architecture diagram of the application including all peripherals, FreeRTOS tasks and queues.

User Interface:

The provided initial user interface is produced by a FreeRTOS task that drives the UART.

A. Motor Parameter Setup:

1. The interface begins by displaying **"Stepper motor initialization complete! Operational parameters can be changed below:"**.
2. The interface will display a series of four parameters (current position, rotational speed, rotational acceleration, and rotational deceleration) with their default values.
3. The user can either press <ENTER> to keep the default value or enter a positive value from the SDK terminal by typing xxx<ENTER> to change a parameter.

```
Stepper motor Initialization Complete! Operational parameters can be changed below:

Current position of the motor = 0 steps
Press <ENTER> to keep this value, or type a new starting position and then <ENTER>
User chooses to keep the default value of current position = 0 steps

Current maximum speed of the motor = 500.0 steps/sec
Press <ENTER> to keep this value, or type a new maximum speed number and then <ENTER>
User chooses to keep the default value of rotational speed = 500.0 steps/sec

Current maximum acceleration of the motor = 150.0 steps/sec/sec
Press <ENTER> to keep this value, or type a new maximum acceleration and then <ENTER>
User chooses to keep the default value of rotational acceleration = 150.0 steps/sec/sec

Current maximum deceleration of the motor = 150.0 steps/sec/sec
Press <ENTER> to keep this value, or type a new maximum deceleration and then <ENTER>
User chooses to keep the default value of rotational deceleration = 150.0 steps/sec/sec

Destination position of the motor = 2048 steps
Press <ENTER> to keep this value, or type a new destination position and then <ENTER>
User entered the new destination position = 4096 steps

Current dwell time of the motor between movements = 0 ms
Press <ENTER> to keep this value, or type a new dwell time and then <ENTER>
```

B. Destination and Delay Setup (Maximum of 10 Pairs):

1. The user is asked to enter a destination position. The initial default value is 2048 steps (1 rotation clockwise).
2. The user can press <ENTER> to keep the default value or type `xxx<ENTER>` to change the destination/target position.
    a. A positive value indicates clockwise rotation, and a negative value indicates counter-clockwise rotation.
3. After the destination position is entered, the user interface displays another message line asking for a delay at that position. The default delay is 0 milliseconds.
4. The user can press <ENTER> to keep the default delay or type xxx<ENTER> to change it.
5. Repeat steps 1-4. The user can specify a maximum of 10 <position, delay> commands. An example of three <destination, delay> pairs is shown below (in the original document).
6. Once all pairs are entered, press <ENTER> again to stop entering more values.

```
User entered new dwell time = 1000 ms
*** Pair: 1    , <destination, delay> = <4096, 1000>

Position slots available: 9
Press <ENTER> to stop entering more positions, or enter a new position and then <ENTER>
User entered the new destination position = -2048 steps

Current dwell time of the motor between movements = 0 ms
Press <ENTER> to keep this value, or type a new dwell time and then <ENTER>
User entered new dwell time = 1000 ms
*** Pair: 2    , <destination, delay> = <-2048, 1000>

Position slots available: 8
Press <ENTER> to stop entering more positions, or enter a new position and then <ENTER>
User entered the new destination position = 6144 steps

Current dwell time of the motor between movements = 0 ms
Press <ENTER> to keep this value, or type a new dwell time and then <ENTER>
User entered new dwell time = 5000 ms
*** Pair: 3    , <destination, delay> = <6144, 5000>

Position slots available: 7
Press <ENTER> to stop entering more positions, or enter a new position and then <ENTER>
User chooses to stop inputting sequences, or sequence full.


****************************** MENU ******************************
1. Press m<ENTER> to change the motor parameters again.
2. Press g<ENTER> to start the movement of the motor.
```

C. Motor Operation:

1. The interface will now ask the user to press m<ENTER> to change the motor parameters again or g<ENTER> to start the motor movement.
2. After the last destination has been reached, the motor should stop.

Stepper Motor Driver Functions:

A library of stepper motor control functions is provided with the Lab 4 software package. Unzip the Lab 4 files to a convenient directory and study the contents of the file's stepper.h and stepper.c.

The header file contains #include directives that specify other required include files, as well as #define definitions of symbolic constants, and prototype headers for the functions that are defined in the accompanying C source file. The stepper motor driver uses the full-step drive waveforms.

The status of the stepper motor is provided by variables including the **currentPosition_InSteps**, **targetPosition_InSteps**, **decelerationDistance_InSteps**. Calculations are made in real-time using variables of type float to update the planned ideal rotor speed, rotor acceleration (or deceleration), and step period. Note that step period resolution must be reduced to 1 millisecond because the tick clock runs at 1 KHz. In stepper.c locate the following useful functions:

```
• void Stepper_Initialize(); // Called to initialize the stepper motor driver

• void Stepper_setCurrentPositionInSteps(int); // Redefines the current position in steps

• void Stepper_setMaxSpeedInStepsPerSecond(float); // Set max allowed speed

• void Stepper_setAccelerationInStepsPerSecondPerSecond(float); // Set max acceleration

• void Stepper_setDecelerationInStepsPerSecondPerSecond(float); // Set max deceleration

• void Stepper_moveToPositionInSteps(int); // Move the motor to a new absolute position

• _Bool Stepper_motionComplete(); // Is the motor at the target position?

• void Stepper_disableMotor(); // Disable the motor
```

PART 1 - UPGRADING THE CONTROLLER — Marks 30%

For this part, students are required to complete the **Task_Motor()** function. This function is responsible for calling the necessary stepper motor control functions to make the motor move. To begin, update the Board Support Package (BSP) to set the tick_rate to 1000. This modification will make the system more responsive, which will allow for better and more precise control of the motor. Following this, you must add the standard math library to the compiler flags. To do this, open the compiler settings (specifically the **UserConfig.cmake** file) using the vitis explorer, navigate to the libraries section, and add **"m"** to your project libraries.

Please enter your code in the commented section inside this task to produce the correct motor rotations. This task is responsible for reading the motor's parameters in the **FIFO1** from **_Task_Uart()**. The functions provided in the aforementioned list will be used to set the maximum rotational speed, rotational acceleration, rotational deceleration, and current position of the motor in steps. After the motor reaches the target position, it is disabled and then it stays in place for the delay period specified in the <destination, delay> pair. The RGB led needs to be turned on with the Green color when the motor initiates movement and should be turned OFF when the motor reaches the target position. The LED should remain ON throughout the entire motor motion. The led should be green again when the motor starts moving towards the second destination. It should go OFF when the target position is reached. The process repeats until all the destination delay pairs have been executed. After that, the LED remains OFF. Students have the ability to specify distinct maximum speed, acceleration and deceleration values each time a fresh set of <destination, delay> pairs is entered. Experiment with various values for speed, acceleration and deceleration to observe how they affect the motor's movement. Create a table by conducting the following experiment with the motor movement.

Experiment: Maximum Acceleration and Deceleration both must be set to 150 steps/sec/sec. Vary the maximum speed of motor from 200 steps/sec to 500 steps/sec in increments of 50 steps/sec. For each speed, record the time it takes to complete one rotation. Explain the reasoning behind the values obtained.

Tip: You can add a variable to the app to automatically keep track of the time it takes to execute each command.

PART 2 Emergency Stop — Marks 30%

In some motor applications, it is essential to have the means to rapidly stop the movement controlled by the motor (this could be a soft soft or a hard stop depending on the system). In this exercise you are to implement a soft emergency stop function using one of the Zybo Z7 board's pushbuttons.

Step 1: The FreeRTOS task **_Task_Emerg_Stop()** has been created for you. If push button BTN0 is pressed for three consecutive polling operations (to prevent bounce) in a row, then call a stepper motor driver function that causes the motor to immediately begin decelerating to a stop. There is a suitable function already present in the stepper.c file that helps to adjust the current position of the motor such that it begins decelerating.

Step 2: When the emergency stop command has been issued, then the RGB LED must blink the red color on the Zybo Z7 board at a frequency of 3 Hz. Implement this operation using the same task that polls the push-button. If an emergency stop button is pressed while the motor is moving, the green color will be replaced immediately by the flashing red color.

Step 3: Verify that the emergency stop condition (stopped motor and flashing red LED) persists until the user performs a hard reset from the FPGA board.

You will demonstrate to a TA both (1) the functioning of your emergency stop command, and (2) the ability to load and execute a sequence of motor positions and dwelling delays.

QUESTIONS

1. Briefly describe the operational differences between the wave drive, full step, and half step modes in stepper motors. How does each mode affect the motor's torque and resolution?

2. Why is it important to control the acceleration and deceleration of a stepper motor in a motion control system? Discuss the potential effects on the system if these factors are not properly managed.

3. Explain how an optoisolator circuit protects sensitive components in a control system. Use the circuit you assembled as a reference.

4. Discuss the role of resistors in the optoisolator circuit connected to the stepper motor. Why are different resistor values used on the input and output sides of the optoisolator?

5. Explain what the slew rate of a stepper motor is and describe a method to experimentally determine it in case it's not provided in the motor's datasheet. Detail the steps you would take to measure the slew rate accurately.

6. Describe a specific problem you encountered during the lab and explain the steps you took to resolve it. How did this issue and its resolution contribute to your understanding of computer interfacing?

RESOURCES

- FreeRTOS Documentation.
- Zybo Z7 Reference Manual.
- LTV-847 Optoisolator Datasheet.
- ULN2003 Darlington Array Datasheet.
- 28BYJ-48 Stepper Motor Datasheet

REPORT REQUIREMENTS

Cover Page:

- ➔ Include course and lab section numbers, lab number, and due date.
- ➔ List all student names in your lab team. Please omit ID numbers to respect privacy.

Abstract:

- ➔ Summarize the lab's objectives, the hardware used and the exercises in your own words.

Design Section:

- ➔ Outline your solutions with a concise explanation of your code for each exercise.
- ➔ Include task flow diagrams. Avoid big sections of code snapshots.

Testing Suite:

- ➔ Provide a table with test descriptions, expected and actual results and a rationale for each test case.

Conclusion:

- ➔ Assess if the given objectives were met and summarize the lab's findings.
- ➔ Discuss any issues if the lab was incomplete or unsuccessful. What were the cause(s) of the issue and how could you resolve them if you had more time.

Formatting and Submission:

- ➔ Submit the report to eCLass as a single PDF file with lab number and student names in the filename.
- ➔ Ensure correct spelling and grammar.
- ➔ Use a readable font size (larger than size 10).
- ➔ Diagrams may be hand-drawn with a straightedge and scanned at 300 pixels per inch resolution.
- ➔ Source code in your report should be syntax-highlighted to enhance readability. Syntax highlighting differentiates code elements such as keywords comments and strings.
- ➔ Adopt a professional format with consistent headings and subheadings.
- ➔ Only submit your modified source code files in one zip folder. Do not zip the entire workspace.
- ➔ Adhere to the submission deadlines as indicated on the first page of the lab handout.

MARKING SCHEME

Students will be graded based on the form and general quality of the report (clarity, organization, tidiness, spelling, grammar) for the lab. The Pre-lab work is worth 10%, the lab demos are worth 60%, the report is worth 30%.